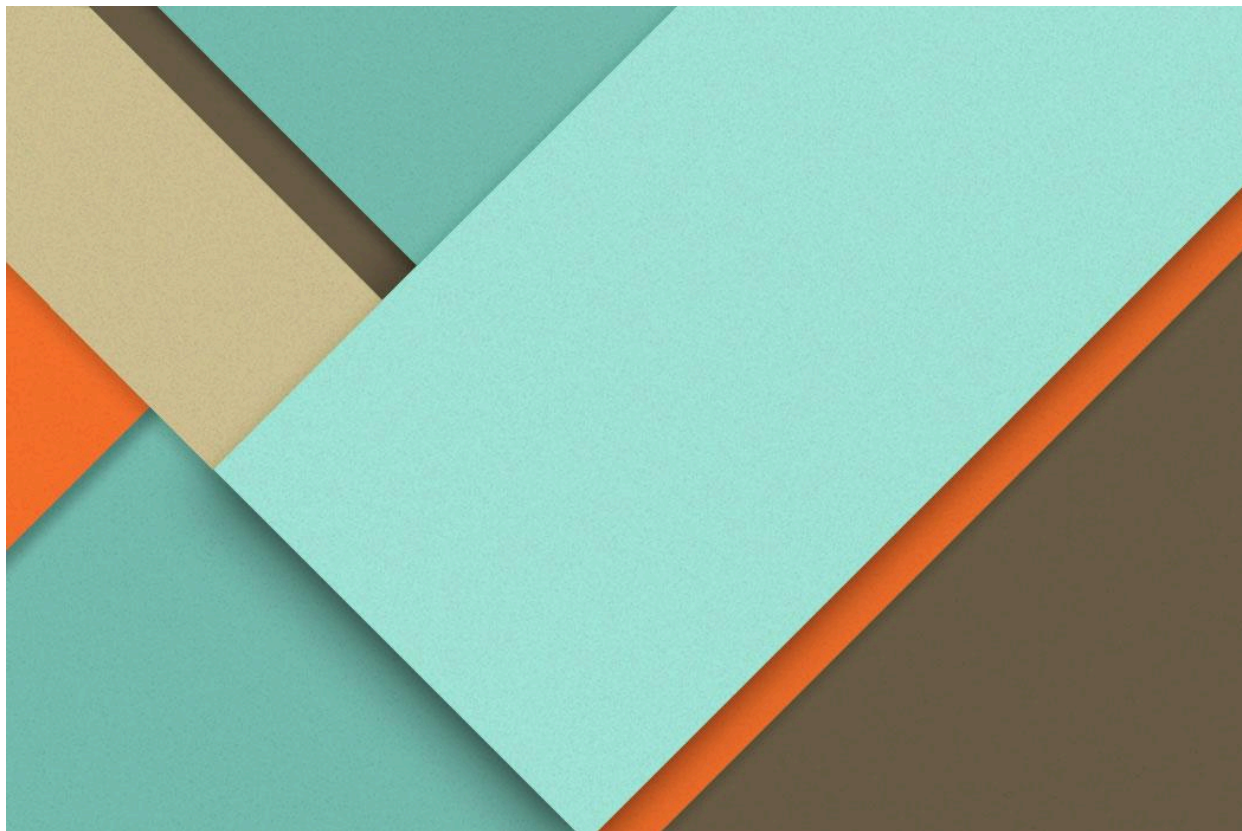




AF5: Tarea Dual ApiRest PSP

22/03/2026

—

José Manuel Sánchez Rosal

2º DAM

IES Antonio Gala

1400 Palma del Río (Córdoba)

1 Índice

1 Índice.....	1
2 Introducción y Arquitectura del Proyecto.....	1
3 Desarrollo del Servicio REST (Cumplimiento RA4).....	2
3.1. Endpoints y Operaciones Básicas (CRUD).....	2
3.2. Concurrencia y Disponibilidad.....	2
4 Seguridad y Protección de Datos (Cumplimiento RA5).....	3
4.1. Autenticación y Esquema Basado en Roles (RBAC).....	3
4.2. Políticas de Control de Acceso.....	3
4.3. Prácticas de Programación Segura y Validación.....	3
5 Pruebas de Funcionamiento y Validaciones.....	4
5.1. Acceso Permitido de Lectura (Rol: USER / ADMIN).....	4
5.2. Operación de Escritura Autorizada (Rol: ADMIN).....	5
5.3. Acceso Denegado por falta de privilegios (Rol: USER).....	6
5.4. Anexo: Gestión Integral de Códigos de Estado HTTP (RA4 y RA5).....	6
6 Conclusiones.....	9

2 Introducción y Arquitectura del Proyecto

Este documento detalla el diseño, desarrollo y securización de una API REST para la gestión de un inventario de productos. El proyecto ha sido desarrollado utilizando **Spring Boot 3.2.2** y **Java 21**, implementando una arquitectura MVC y asegurando los endpoints mediante **Spring Security**.

El objetivo principal es demostrar la adquisición de las competencias de los Resultados de Aprendizaje referentes a servicios en red (RA4) y protección de aplicaciones (RA5). El servicio se despliega de forma autónoma sobre un servidor Tomcat embebido en el puerto 8080.

3 Desarrollo del Servicio REST (Cumplimiento RA4)

Se ha implementado una API REST completa capaz de gestionar múltiples clientes concurrentes de forma segura y eficiente. La lógica de negocio y exposición de endpoints se encuentra en la clase `ProductoController`.

3.1. Endpoints y Operaciones Básicas (CRUD)

Se han desarrollado las cuatro operaciones fundamentales sobre la ruta base `/api/productos`:

- **Listar (GET):** Devuelve la colección completa de productos.
- **Añadir (POST):** Recibe un objeto JSON, autoincrementa su ID y lo inserta en el sistema, devolviendo un estado `201 Created`.
- **Modificar (PUT):** Recibe un ID por ruta (`/id`) y un JSON por cuerpo para actualizar los datos, devolviendo un estado `200 OK` si existe, o `404 Not Found` en caso contrario.
- **Eliminar (DELETE):** Elimina el recurso asociado al ID proporcionado en la URL.

3.2. Concurrencia y Disponibilidad

En un entorno de red, múltiples peticiones pueden llegar simultáneamente. Para evitar errores de inconsistencia y condiciones de carrera (Race Conditions), se han aplicado las siguientes estrategias en la memoria de la aplicación:

- **Colecciones Thread-Safe:** El inventario se almacena en una lista `CopyOnWriteArrayList`, la cual evita excepciones de concurrencia si varios hilos intentan leer y escribir al mismo tiempo.
- **Atomicidad en IDs:** La generación de identificadores únicos se gestiona mediante la clase `AtomicInteger` (`contadorIds`), garantizando una asignación segura sin bloqueos.

4 Seguridad y Protección de Datos (Cumplimiento RA5)

La API ha sido protegida para garantizar que solo usuarios autorizados puedan consultar o alterar la información. Las directivas de seguridad se centralizan en la clase `SecurityConfig`.

4.1. Autenticación y Esquema Basado en Roles (RBAC)

Se ha implementado un sistema de autenticación HTTP Basic almacenando los usuarios en memoria mediante `InMemoryUserDetailsManager`. Se han definido dos perfiles con privilegios distintos:

- **Rol USER:** Usuario estándar ("`user`" / "`1234`") diseñado únicamente para la consulta de información.
- **Rol ADMIN:** Administrador ("`admin`" / "`admin`") con control total sobre el inventario.

4.2. Políticas de Control de Acceso

Aplicando el principio de mínimo privilegio, se han restringido las rutas en el `SecurityFilterChain`:

- Ningún endpoint es público; toda petición requiere estar autenticado (`anyRequest().authenticated()`).
- Las peticiones de lectura (`GET /api/productos/**`) están permitidas para cualquier usuario autenticado.
- Las operaciones de escritura y destrucción (`POST, PUT, DELETE`) están estrictamente limitadas a los usuarios con el rol `ADMIN`.

4.3. Prácticas de Programación Segura y Validación

Para evitar la inyección de datos corruptos, el controlador implementa el método de validación `esValido(Producto producto)`. Este método asegura que no se procesen peticiones `POST` o `PUT` cuyo campo "nombre" sea nulo o esté compuesto únicamente por espacios en blanco. Si la validación falla, el servidor aborta la operación y responde inmediatamente con un código `400 Bad Request`.

5 Pruebas de Funcionamiento y Validaciones

Se ha verificado el correcto funcionamiento del ciclo de vida CRUD y la seguridad mediante el control de acceso basado en roles (RBAC) sobre los endpoints desarrollados. A continuación se presentan las evidencias de las pruebas:

5.1. Acceso Permitido de Lectura (Rol: USER / ADMIN)

Ambos roles tienen permisos para consultar el catálogo. El sistema devuelve un estado **200 OK**.

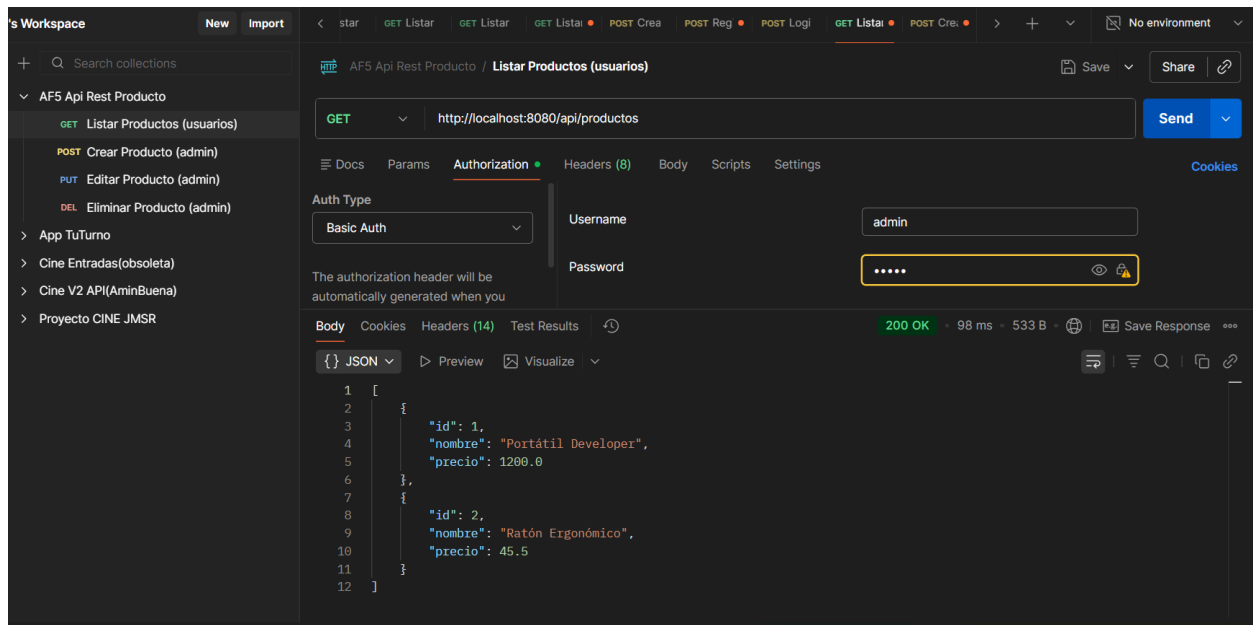
Prueba de listado con credenciales USER:

The screenshot shows a REST client interface with the following details:

- Collection:** AF5 Api Rest Producto / Listar Productos (usuarios)
- Method:** GET
- URL:** http://localhost:8080/api/productos
- Auth Type:** Basic Auth
- Username:** user
- Password:** [Redacted]
- Status:** 200 OK
- Response Body (JSON):**

```
1 [
2   {
3     "id": 1,
4     "nombre": "Portátil Developer",
5     "precio": 1200.0
6   },
7   {
8     "id": 2,
9     "nombre": "Ratón Ergonómico",
10    "precio": 45.5
11  }
12 ]
```

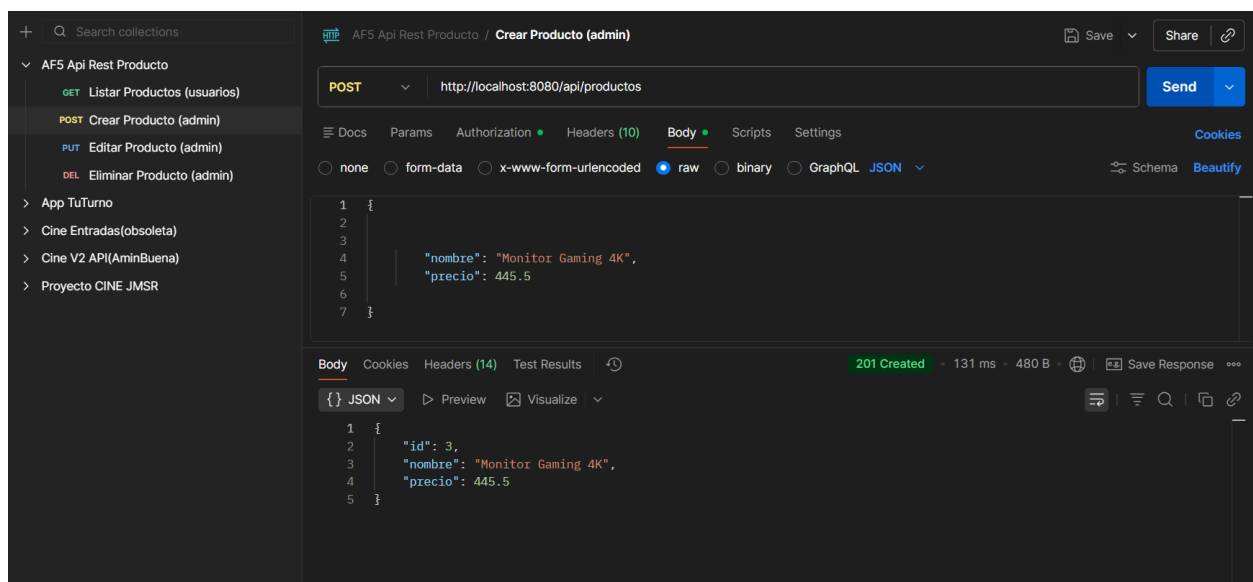
Prueba de listado con credenciales ADMIN:



5.2. Operación de Escritura Autorizada (Rol: ADMIN)

El administrador realiza una petición **POST** válida. El sistema la procesa, autoincrementa el ID de forma atómica y concurrente, y devuelve el recurso creado con el estado **201 Created**.

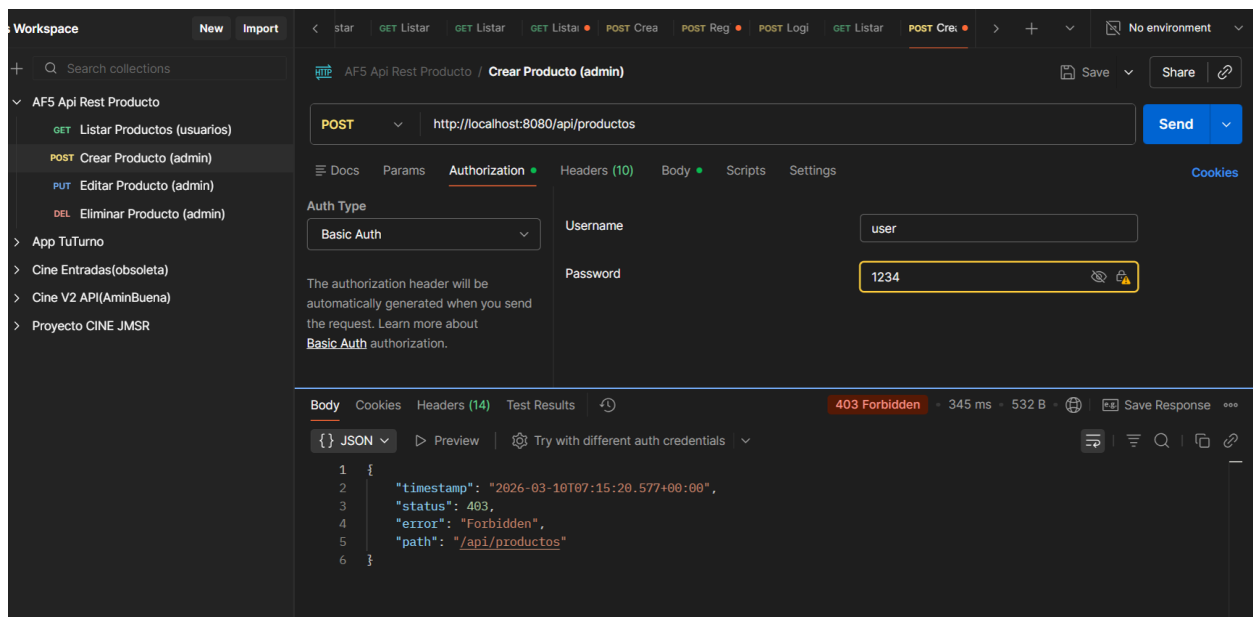
Prueba de creación con credenciales ADMIN:



5.3. Acceso Denegado por falta de privilegios (Rol: USER)

El usuario con rol **USER** intenta ejecutar una operación restringida (añadir un producto mediante **POST**). El filtro de Spring Security intercepta la petición antes de llegar al controlador, denegando la acción y devolviendo correctamente un estado **403 Forbidden**.

Prueba de operación denegada con credenciales USER:



5.4. Anexo: Gestión Integral de Códigos de Estado HTTP (RA4 y RA5)

Para garantizar el cumplimiento riguroso de los protocolos estándar HTTP (RA4.a) y una correcta política de seguridad y validación (RA5.j, RA5.h), esta API devuelve códigos de estado semánticos dependiendo del resultado de la operación y los permisos del cliente:

Código HTTP	Estado	Familia Concepto /	Significado Técnico	Ejemplo en este proyecto
200	OK	Éxito	La petición se ha procesado correctamente y el servidor devuelve los datos solicitados.	Un GET exitoso para listar el catálogo o un PUT que modifica un producto existente.
201	Created	Éxito (Creación)	La petición ha sido exitosa y ha resultado en la creación de un nuevo recurso en el servidor.	Un POST válido realizado por el ADMIN que genera un nuevo ID en el inventario.
400	Bad Request	Error de Cliente (Validación)	El servidor no puede procesar la petición porque la sintaxis es inválida o los datos no cumplen los requisitos.	Intentar hacer un POST o PUT enviando un producto con el campo "nombre" vacío o nulo.
401	Unauthorized	Seguridad (Identidad)	El cliente no ha provisto credenciales válidas o la cabecera de	Intentar acceder a cualquier ruta de /api/productos sin configurar la pestaña

			autenticación está ausente.	<i>Authorization</i> en Postman.
403	Forbidden	Seguridad (Autorización)	El cliente está autenticado correctamente, pero su rol no tiene privilegios suficientes para esa acción.	Un usuario con rol USER intentando hacer un DELETE o POST (acciones exclusivas de ADMIN).
404	Not Found	Error de Cliente (Recurso)	El servidor no puede encontrar el recurso solicitado en la URL especificada.	Intentar modificar (PUT) o borrar (DELETE) un producto con un ID que no existe en el inventario (ej. ID 99).

6 Conclusiones

- **Robustez en Servicios de Red (RA4):** El desarrollo de la API REST utilizando Spring Boot 3.2.2 y Java 21 ha resultado en la creación de un servicio eficiente que maneja adecuadamente el ciclo de vida completo de las operaciones CRUD.
- **Gestión Eficaz de la Concurrencia:** La implementación estratégica de estructuras *Thread-Safe*, concretamente [CopyOnWriteArrayList](#), junto con la clase [AtomicInteger](#) para la generación de identificadores, garantiza la integridad de los datos y previene las condiciones de carrera ante peticiones simultáneas de múltiples clientes.
- **Protección y Control de Acceso (RA5):** La integración de Spring Security ha permitido centralizar y blindar el acceso a los recursos. El sistema de Autenticación Basic con un enfoque de Control de Acceso Basado en Roles (RBAC) cumple estrictamente con el principio de mínimo privilegio, diferenciando de forma efectiva las capacidades de lectura del rol [USER](#) frente a los permisos totales del rol [ADMIN](#).
- **Prácticas de Programación Segura:** La inclusión de métodos de validación explícitos para evitar la inyección de datos nulos o vacíos en las peticiones demuestra una aplicación proactiva de la seguridad a nivel de código.
- **Cumplimiento de Estándares Web:** Las pruebas realizadas han verificado que el sistema no solo bloquea accesos no autorizados, sino que también emite códigos de estado HTTP semánticamente precisos (como [201 Created](#), [400 Bad Request](#) o [403 Forbidden](#)), asegurando una comunicación estandarizada y predecible con el cliente.